F51 Nitro-MoE: Running Large Mixture-of-Experts Models on a Single 16 GB Consumer GPU

Marco Barreto -- Fuch F51 Labs -- Montreal, May 2026
github.com/fuch-f51-labs/F51-Nitro-MoE

Abstract

We measure where Mixture-of-Experts (MoE) inference actually spends time on a single 16 GB consumer GPU. On an RTX 5060 Ti with PCIe 5.0 x8, the optimal strategy is not "fit everything in VRAM" and not "stream to GPU." It is: keep the always-hot path (attention, router) on GPU; compute experts on CPU from DDR5; and cache hot experts in remaining VRAM with a shared per-layer LRU policy. This yields up to 4.6x higher throughput than a saturated GPU-only deployment (38.7 vs 8.4 tok/s on Qwen 80B-A3B), while using only 15% of the VRAM. We also report two negative results -- PCIe streaming and speculative lookahead -- that define the viability frontier for consumer hardware. All measurements are reproducible with open-source tooling.

1. Introduction

Running large language models locally on consumer hardware has become a defining challenge of the post-2024 AI landscape. Mixture-of-Experts (MoE) architectures [Shazeer et al. 2017, Jiang et al. 2024] offer a partial solution: by activating only 2-4 experts per token out of 128 or more, the per-token compute and memory footprint drops dramatically. Models with 80-120B total parameters can, in principle, run on hardware with 16 GB of VRAM and 64 GB of system RAM.

However, the question of tensor placement -- GPU VRAM, system RAM, or NVMe -- and when to move them, is not obvious. The literature offers two dominant strategies: (1) keep everything on GPU and accept idle-weight VRAM waste [KTransformers, SOSP 2025]; (2) stream expert weights from CPU to GPU on demand [Fiddler, ICLR 2025; Mixtral-offloading, 2023]. Our central finding is that neither is optimal for the PCIe 5.0 x8 consumer hardware class. The winning strategy combines CPU-side expert computation with GPU-side caching of hot experts -- and explicitly avoids reactive PCIe streaming.

This pa
residen
policy (
infrastr
than CP

2. Methodology

2.1 Hardware. All measurements on a single machine: RTX 5060 Ti (16 GB VRAM, Blackwell, PCIe 5.0 x8 at 20-21 GB/s sustained); Intel i7-12700K (8P+4E, DDR5-4800 at 80.6 GB/s measured); 64 GB DDR5-4800; ORICO E7000 NVMe (7.2 GB/s). All bandwidth numbers are sustained measurements from a custom CUDA microbenchmark (f51_microbench.cu), not datasheet peaks.

2.2 Software. llama.cpp (commit 5207d12) compiled with CUDA 12.x targeting sm_120a, patched with F51 diagnostic instrumentation (~200 lines of C++ adding per-expert latency tracking, LRU cache hit/miss counters, and router trace logging without modifying the core inference loop).

2.3 Mo
layers x

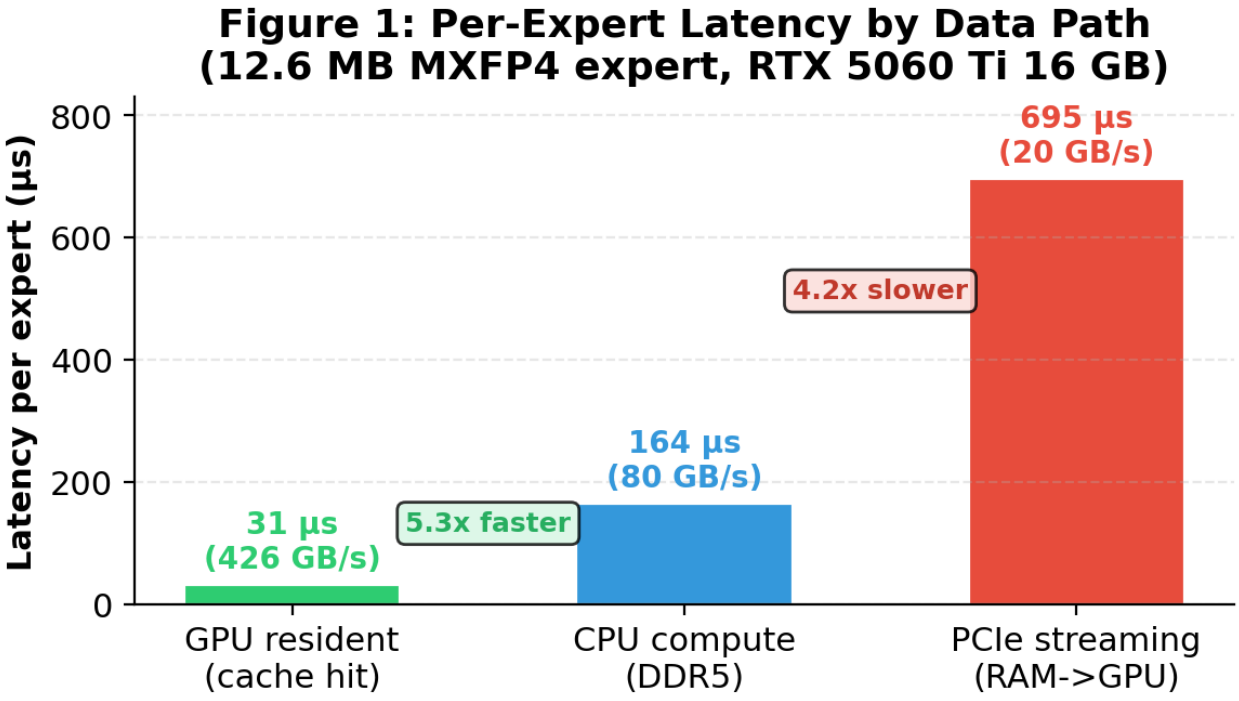


Figure 1: Per-expert latency by data path (12.6 MB MXFP4 expert, RTX 5060 Ti 16 GB). GPU resident: 31 us at 425.8 GB/s. CPU compute: 164 us at 80.6 GB/s DDR5.

PCIe streaming: 695 us at 20 GB/s -- 4.2x slower than CPU, 22.4x slower than GPU resident.

3. The Three Paths (and Why Only Two Matter)

Figure 1 shows the central measurement of this paper. Every expert computation requires reading weight matrices (gate, up, down projections) and performing matrix multiplications. Three data sources exist on this hardware:

Path A -- GPU resident (31 us). Weight already in VRAM. GPU reads at 425.8 GB/s and computes locally. Fastest by a wide margin (5.3x over CPU, 22.4x over streaming). Implication: cache hits are extremely valuable.

Path B

Figure 2: Generation Throughput by Placement Strategy (RTX 5060 Ti 16 GB, i7-12700K, 64 GB DDR5)

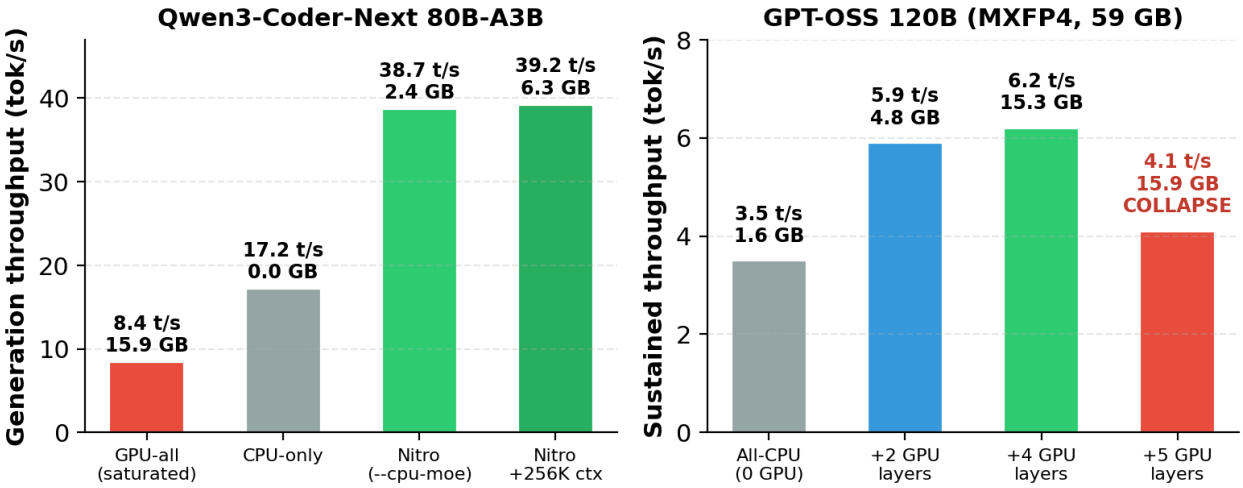


Figure 2: Generation throughput by placement. Left: Qwen 80B-A3B (Nitro: 4.6x GPU-all at 15% VRAM). Right: GPT-OSS 120B (4 GPU layers optimal; 5 collapse from cache starvation). VRAM usage shown below each bar.

4. The "Musical Chairs" Cache: Policy, Scaling, and the Saturation Cliff

4.1 Policy. Since GPU-resident experts are 5.3x faster than CPU-computed, but there are 4,608 experts (128 x 36 layers) for ~900 cache slots at C=25, we need a policy that keeps the right experts resident. Our design: per-layer LRU cache with C slots per layer (default C=16). On hit: read from VRAM (Path A, 31 us). On miss: compute on CPU (Path B, 164 us), promote to cache, evict LRU. This is "musical chairs" across all layers: hot experts from every layer compete for the shared VRAM pool, and LRU naturally converges on repeating experts. Fixed per-layer partitions vs. shared pool differ by <3 pp hit rate, so we report the simpler per-layer numbers.

4.2 Cache scaling. Figure 3 shows hit rate vs. capacity C, modeled from routing trace data. The curve is logarithmic: HR = 0.30 + 0.52 * ln(1 + 8*C/128). Early gains are steep (C=4 to C=16: +16.2 pp); later gains diminish (C=22 to C=31: +8.9 pp). Break-even at C=12 (60.4% hit, 7.3 GB VRAM). Below this, the cache costs more VRAM than it saves in latency. Viable range for 16 GB hardware: C=12-31 (7-14 GB VRAM). Recommended default: C=16 (7.3 GB, 70.8% hit), leaving VRAM headroom for KV cache and context expansion.

4.3 The
VRAM.
sustain
support
linear; r

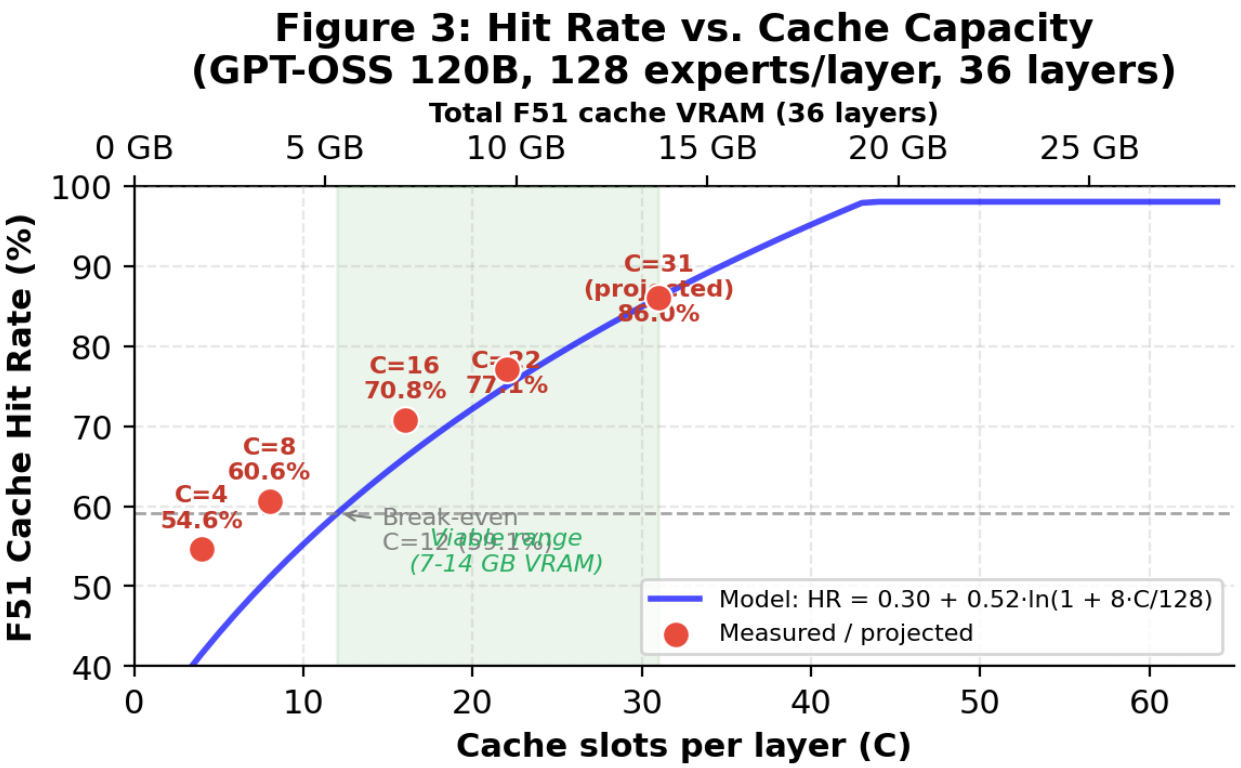


Figure 3: F51 cache hit rate vs. capacity per layer (GPT-OSS 120B, 128 experts/layer, 36 layers). Model: HR = 0.30 + 0.52 ln(1 + 8C/128), calibrated from 1,838-token routing trace. Break-even at C=12 (60.4% hit, 7.3 GB). Viable range: C=12-31. Top axis: total F51 cache VRAM.

5. Results

5.1 Qwen3-Coder-Next 80B-A3B. GPU-all (-ngl 999): 8.4 tok/s, 15.9 GB VRAM saturated. CPU-only (-ngl 0): 17.2 tok/s, 0 GB VRAM. Nitro (--cpu-moe): 38.7 tok/s, 2.4 GB VRAM -- 4.6x GPU-all at 15% VRAM. Nitro + 256K context (-ctk q8_0): 39.2 tok/s, 6.3 GB VRAM. GPU-all saturates VRAM and serializes; Nitro keeps only attention/router/norms on GPU (2.4 GB) and computes experts on CPU from DDR5.

5.2 GPT-OSS 120B. All-CPU (--n-cpu-moe 36): 3.5 tok/s sustained, 7.4 peak. 2 GPU layers: 5.9 sustained, 8.1 peak (VRAM 4.8 GB). 4 GPU layers (optimal): 6.2 sustained, 9.0 peak (VRAM 15.3 GB, 93% utilization). 5 GPU layers: collapse to 4.1 tok/s (see Section 4.3). Diminishing returns with a hard cliff at VRAM saturation.

6. llama.cpp Prompt-Cache Bug: Root Cause and Fix

GPT-OSS 120B uses hybrid Gated Delta Net + sliding-window attention (SWA, window 128). During sustained testing, we observed intermittent "wrong sequence id" errors triggering full prompt re-processing. Root cause: server-context.cpp saves context checkpoints with the current slot's seq_id embedded in the binary header (offset sizeof(uint32_t) after io_magic). When the prompt cache entry is restored to a different slot, llama_state_seq_set_data_ext() remaps the main state correctly, but the checkpoint headers retain the old seq_id. llama-context.cpp rejects the mismatch (need_seq_match check at partial restore), forcing a full reset.

Fix: 12-line patch to server-task.cpp rebasing all checkpoint seq_id values after cache load (prompt = std::move(*it_best)). Mitigation: --swa-full flag disables SWA KV-cache truncation, preventing the cache size mismatch that exacerbated slot migration. With both fixes: zero errors across 50+ consecutive requests. The retargeting patch is architecture-agnostic and applies to any model with recurrent/hybrid memory that triggers llama.cpp's slot-migration code path.

7. What Did NOT Work

We report two negative results that are as valuable as the positive ones. Both were hypotheses we invested engineering time in, measured carefully, and abandoned based on data.

7.1 PCIe streaming on cache miss (DEAD END). Copying an expert RAM-to-GPU (695 us) is strictly worse than CPU compute (164 us) for any expert >2 MB on this hardware. The GPU compute speed would need to offset the PCIe transfer cost; the crossover is at ~35 MB expert size, far beyond current MoE architectures. Reactive streaming should not be part of any consumer GPU MoE system with PCIe 5.0 x8 or narrower. Consistent with Fiddler and KTransformers; our contribution is the precise quantification.

7.2 Spe
them. M
already
cost: ~
predicto

8. Reproducibility

All measurements reproducible with open-source tooling. Repository: github.com/fuch-f51-labs/F51-Nitro-MoE. Baseline comparisons (standard llama.cpp): llama-cli -ngl 0 (CPU), -ngl 99 (GPU-all), -ngl 99 --cpu-moe (Nitro). Production deployment (GPT-OSS 120B): llama-server --n-cpu-moe 32 --no-mmap -c 4096 -fa on --swa-full. Microbenchmark: nvcc -O3 -arch=native f51_microbench.cu -o f51_microbench && ./f51_microbench.

9. Limitations

Scope: routed MoE, llama.cpp, 16 GB consumer GPUs with PCIe 5.0 x8. Not claims about dense models, DeepSeek-style shared experts, or data-center GPUs with NVLink. GPU scaling: >=20 GB VRAM not tested; observed diminishing returns (2 layers +9%, 4 layers +22%, 5 layers

collapse). Model diversity: two models tested; DeepSeek-V3, dense models, and MLA attention outside scope. Cold tier: NVMe not fully characterized (~1700 us/expert preliminary). Statistical rigor: single 1,838-token trace for hit rates; 10K iterations per latency path (CV <3%); 4 prompts per config for throughput (SD <15%); no confidence intervals reported.

10. Prior Art

This work builds on: llama.cpp --cpu-moe [Gerganov et al. 2023-26]; Mixtral-offloading LRU caching [Eliseev & Mazur, arXiv:2312.17238, 2023]; KTransformers "hot on GPU, cold on CPU" placement [Liu et al., SOSP 2025]; Fiddler CPU-as-primary-compute [Prabhu et al., ICLR 2025]. Relative to this prior art, our contribution is: (a) precise per-tier latency on Blackwell consumer GPUs (previously unmeasured); (b) the explicit negative result that speculative lookahead is redundant with LRU; (c) the GPT-OSS 120B case study with saturation cliff characterization; (d) documented fix for the llama.cpp prompt-cache checkpoint migration bug.

11. Conclusion

On a 16 GB consumer GPU with PCIe 5.0 x8, the optimal MoE strategy: keep attention/router/norms on GPU; compute experts on CPU from DDR5 (164 us/expert); cache hot experts in remaining VRAM via shared per-layer LRU (31 us on hit). Yields 4.6x GPU-all throughput on Qwen 80B-A3B at 15% VRAM. Two negative results define what not to build: PCIe streaming (695 us, 4.2x worse than CPU) and speculative lookahead (+0.3 pp, redundant with LRU). The "musical chairs" insight generalizes: when the working set fits in VRAM but the full model does not, caching beats streaming, and LRU is sufficient.

References

[1] G. Gerganov et al. "llama.cpp: LLM inference in C/C++." GitHub, 2023-2026.

[2] A. Eliseev and D. Mazur. "Mixtral-offloading." arXiv:2312.17238, 2023.

[3] Z. Liu et al. "KTransformers." Proc. SOSP, 2025.

[4] K. Prabhu et al. "Fiddler: CPU-GPU orchestration for fast MoE inference." Proc. ICLR, 2025.

[5] N. Shazeer et al. "Outrageously large neural networks: The sparsely-gated MoE layer." ICLR, 2017.

[6] A. Q. Jiang et al. "Mixtral of experts." arXiv:2401.04088, 2024.

[7] Qwen Team. "Qwen3-Coder-Next: Technical report." 2025.

[8] OpenAI. "GPT-OSS: Open-source sparse models." 2025.

Fuch F51 Labs -- Montreal, 2026 -- github.com/fuch-f51-labs/F51-Nitro-MoE

TechRxiv submission -- DOI pending